

A3-PS003-TC242

Federated Traffic-Flow Prediction with Communication-Cost Evaluation

Pratik Sen pratiksen2405@gmail.com
Niladri Paul nilpaul1601@gmail.com

Mentor: Partha Chakraborty (Partha.Chakraborty@iem.edu.in)

AIORI Hackathon 3

October 5, 2026

Problem Description

- Centralised traffic forecasting needs roadside sensors and edge cameras to stream raw data to a cloud: **high cellular bandwidth** and **concentrated private travel logs**.
- Federated Learning (FL) trains models on-device and shares only **weight updates** with a server.
- We build a **federated simulator** on a public traffic dataset partitioned across 3–5 virtual intersections (clients).
- We evaluate how a compact forecaster trained with **FedAvg** copes with **non-IID** traffic patterns, reporting **MAE**/**RMSE** against **bytes transmitted per round**.
- Scope: simulation only—no live cameras, no city-wide deployment, no formal privacy guarantees.

Background

- **Traffic forecasting** predicts future flow/speed from past sensor readings; recurrent models (LSTM/GRU) capture temporal dependence [6, 7].
- **Federated Averaging (FedAvg)**: each round the server sends global weights, clients train locally for E epochs, and the server averages returned weights in proportion to local sample counts n_k [1]:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k$$

- **Non-IID data**: different intersections see different volumes and peak patterns, which slows or destabilises convergence [4, 5].
- **Communication cost** per round is $\approx 2\,K\,|w| \cdot 4$ bytes (upload + download, float32), so a compact model directly lowers network load.

Literature & RFC Review

- **McMahan et al. (2017)** – introduces FedAvg and shows large reductions in communication rounds versus synchronised SGD; our core algorithm [1].
- **Konečný et al. (2016)** – strategies for communication-efficient FL; motivates reporting bytes per round [2].
- **Li et al. (2018), DCRNN** – establishes METR-LA as a standard traffic benchmark with MAE/RMSE evaluation [3].
- **Hsu et al. (2019)** and **Li et al. (2020), FedProx** – quantify the effect of non-identical client data on FedAvg convergence [4, 5].
- **Gap addressed**: a compact, reproducible comparison of centralised vs. federated forecasting that jointly reports accuracy, convergence and byte cost under IID vs. non-IID splits across 3 seeds.

Proposed Solution

- **Data**: METR-LA, one city, 3–5 sensor groups as simulated clients; chronological train/validation/test split to avoid leakage.
- **Model**: one compact LSTM/GRU forecaster implemented in PyTorch.
- **Federated engine**: custom Python FedAvg; an **orchestrator agent** coordinates 3–5 **client sub-agents** built with the Google Antigravity SDK (Gemini).
- **Partitions**: IID (samples shuffled across clients) vs. non-IID (each client keeps its own sensors' traffic pattern).
- **Baselines**: centralised training on pooled data vs. federated training.
- **Metrics**: MAE, RMSE, bytes per round, rounds to convergence; every experiment repeated for **3 random seeds**.
- **Delivery**: results logged to SQLite and shown in a Streamlit + Plotly dashboard.

Optimization Proposed

Aspect	Before (centralised)	After (proposed federated)
Data leaving the edge	Raw sensor time series	Model weight updates only
Network cost	Full dataset uploaded continuously	$\approx 2K\|w\| \cdot 4$ bytes per round, measured and logged
Model size	Large, unconstrained	Compact LSTM/GRU to keep $\|w\|$ small
Local work	None (all in cloud)	E local epochs per round to reduce the number of rounds
Heterogeneity	Not applicable	IID vs. non-IID compared explicitly
Experiment control	Manual scripts	Agent orchestration + SQLite logging + dashboard

Solution Architecture & Design

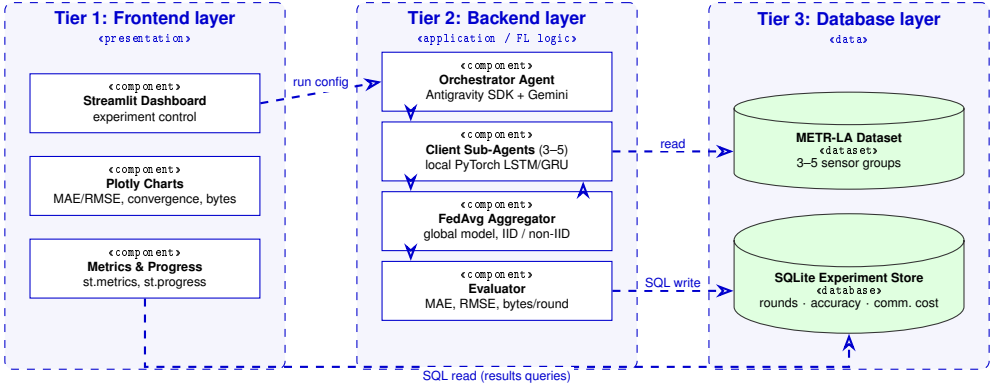


Figure: Three-tier system architecture (UML component view)

Data Flow Diagram

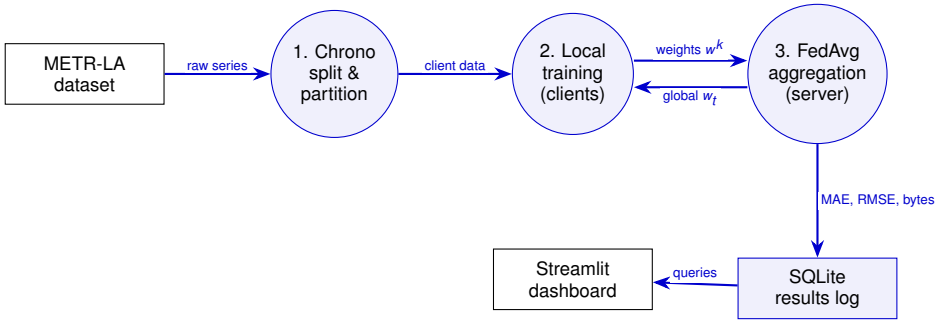


Figure: Data Flow Diagram (DFD). Raw data never leaves a client; only weights move.

Deployment Timeline

- **Day 1** – Streamlit UI shell with hardcoded dummy data.
- **Days 2–3** – Replace dummy data with real FL backend output: PyTorch LSTM/GRU, FedAvg, IID and non-IID splits (GPU training on Google Colab).
- **Day 4** – Add SQLite layer storing rounds, MAE/RMSE and bytes per round for all 3 seeds.
- **Day 5** – Polish, connect end-to-end, final evaluation report, deploy on Streamlit Cloud.
- **Method**: VS Code for Streamlit/PyTorch/FedAvg/SQLite and GitHub commits; Antigravity IDE for agent orchestration; private GitHub repository shared with the mentor throughout.

Bibliography

[1] H. B. McMahan et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data.” In: *AISTATS*, PMLR 54 (2017), pp. 1273–1282.

[2] J. Konečný et al. “Federated Learning: Strategies for Improving Communication Efficiency.” arXiv:1610.05492 (2016).

[3] Y. Li et al. “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting.” In: *ICLR* (2018).

[4] T.-M. H. Hsu et al. “Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification.” arXiv:1909.06335 (2019).

[5] T. Li et al. “Federated Optimization in Heterogeneous Networks.” In: *MLSys* (2020).

[6] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory.” In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[7] J. Chung et al. “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.” arXiv:1412.3555 (2014).

Thank You for Your Attention!